Document 7 — Kaggle Training Workflow, Elaborated Setup & Dataset Checklist

SIH26166 · Addendum to Documents 1 and 3 · Drafted August 31, 2026

1. Your workflow, confirmed: generic Kaggle notebook → export → ARC-AGI-3 notebook as dataset input

Yes — this works, and it is a legitimate, commonly-used pattern on Kaggle. To state it precisely:

- Open a **standard, non-competition-scoped Kaggle notebook** (Settings → Accelerator → GPU T4 x2, or P100). These have full internet access and are not subject to the ARC-AGI-3 competition's no-internet rule.
- Do data prep and model training there: pull PRADAN/LROC/DEM data, generate synthetic pairs, train PatchEncoder, export to descriptor.onnx (see §2 below for the full script).
- Package the exported artifact (the .onnx file, a few MB — not the raw imagery) as a new Kaggle Dataset: kaggle datasets create.
- Open the ARC-AGI-3 competition notebook, **Add Input** → **your dataset**. It mounts read-only at /kaggle/input/<dataset-slug>/, exactly like any other Kaggle Dataset attached to any notebook.
- From there you can load the ONNX weights with onnxruntime inside the ARC-AGI-3 notebook and run inference / further pipeline code against the rtx6000 accelerator, using the dataset as a pre-trained artifact rather than training from scratch inside that notebook.

The one factual point worth flagging, not as an objection but so the GPU choice is deliberate: a standard Kaggle notebook gives you a T4×2 or P100, not the RTX 6000 Pro (96 GB) that the ARC-AGI-3 notebook's rtx6000 accelerator unlocks. For the PatchEncoder triplet-loss model in this project (small, patch-sized inputs, ~128-D embeddings) that difference doesn't matter — T4×2 trains it in minutes. If you ever move to a larger backbone where the RTX 6000's VRAM matters, the alternative in Document 1 §A.3 (train *inside* the ARC-AGI-3 notebook itself, doing data-prep in a separate internet-on pass first, then switching the same notebook to the accelerator) gets you the bigger card. Both are valid; use whichever fits the model size. Since you've done this before and know the mechanics, treat this paragraph as a one-line footnote, not a gate.

2. Elaborated training setup — generic Kaggle notebook (T4×2), full script

2.1 Notebook settings

- kaggle.com → Create → New Notebook (do **not** fork from a competition — this keeps it outside the ARC-AGI-3 competition scope entirely).
- Notebook Settings → Accelerator → **GPU T4 x2** (or P100 if offered). Internet: **On**.
- Persistence: "Files only" is fine — you export the ONNX file at the end and it survives a session restart via Output.

2.2 Cell 1 — install & imports

```
!pip install -q rasterio onnxruntime pds4-tools

import os, glob, random
import numpy as np
import rasterio
import torch, torch.nn as nn, torch.nn.functional as F
```

```
from torch.utils.data import Dataset, DataLoader

DATA_DIR = "/kaggle/working/data"
DEM_DIR  = f"{DATA_DIR}/dem_patches"       # populated in Cell 2
PAIR_DIR = f"{DATA_DIR}/synthetic_pairs"   # populated in Cell 3
CKPT_DIR = "/kaggle/working/checkpoints"
os.makedirs(CKPT_DIR, exist_ok=True)
os.makedirs(PAIR_DIR, exist_ok=True)

DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
print(torch.cuda.is_available(), torch.cuda.device_count(),
      [torch.cuda.get_device_name(i) for i in range(torch.cuda.device_count())])
```

2.3 Cell 2 — attach your DEM patches (uploaded as a Kaggle Dataset, per §3 below)

```
# Attach your cropped DEM patches as a Kaggle Dataset first (Add Input), then:
DEM_SRC = "/kaggle/input/lunar-dem-patches-v1"   # <- your dataset slug
dem_files = sorted(glob.glob(f"{DEM_SRC}/**/*.tif", recursive=True))
print(f"{len(dem_files)} DEM patch files found")
assert len(dem_files) > 0, "Attach the DEM-patches dataset via Add Input before running this cell"
```

2.4 Cell 3 — synthetic sun-angle pair generator (Lommel-Seeliger shading)

```
def lommel_seeliger(albedo, i, e):
    return albedo * np.cos(i) / (np.cos(i) + np.cos(e) + 1e-8)

def render_shaded(dem, sun_azimuth_deg, sun_elevation_deg, albedo=0.12):
    gy, gx = np.gradient(dem)
    normal = np.dstack([-gx, -gy, np.ones_like(dem)])
    normal /= np.linalg.norm(normal, axis=2, keepdims=True)
    az, el = np.radians(sun_azimuth_deg), np.radians(sun_elevation_deg)
    sun_vec = np.array([np.cos(el)*np.cos(az), np.cos(el)*np.sin(az), np.sin(el)])
    cos_i = np.clip(normal @ sun_vec, 0, 1)
    i = np.arccos(cos_i)
    e = np.zeros_like(i)
    return lommel_seeliger(albedo, i, e).astype(np.float32)

def build_triplet(dem_path, other_dem_paths, patch=128):
    with rasterio.open(dem_path) as src:
        dem = src.read(1).astype(np.float32)
    if dem.shape[0] < patch or dem.shape[1] < patch:
        return None
    y = random.randint(0, dem.shape[0]-patch)
    x = random.randint(0, dem.shape[1]-patch)
    crop = dem[y:y+patch, x:x+patch]

    anchor   = render_shaded(crop, random.uniform(0,360), random.uniform(15,70))
    positive = render_shaded(crop, random.uniform(0,360), random.uniform(15,70))

    neg_path = random.choice(other_dem_paths)
    with rasterio.open(neg_path) as src:
        neg_dem = src.read(1).astype(np.float32)
    if neg_dem.shape[0] < patch or neg_dem.shape[1] < patch:
        return None
    ny = random.randint(0, neg_dem.shape[0]-patch)
    nx = random.randint(0, neg_dem.shape[1]-patch)
    neg_crop = neg_dem[ny:ny+patch, nx:nx+patch]
    negative = render_shaded(neg_crop, random.uniform(0,360), random.uniform(15,70))

    def norm(a):
        a = (a - a.min()) / (a.max() - a.min() + 1e-8)
        return a[None, :, :]  # (1, H, W)

    return norm(anchor), norm(positive), norm(negative)

class TripletDataset(Dataset):
    def __init__(self, dem_files, patch=128, length=20000):
        self.dem_files = dem_files
        self.patch = patch
        self.length = length
```

```
    def __len__(self):
        return self.length

    def __getitem__(self, idx):
        path = random.choice(self.dem_files)
        others = [p for p in self.dem_files if p != path] or self.dem_files
        triplet = None
        while triplet is None:
            triplet = build_triplet(path, others, self.patch)
        a, p, n = triplet
        return torch.from_numpy(a), torch.from_numpy(p), torch.from_numpy(n)

dataset = TripletDataset(dem_files, patch=128, length=20000)
dataloader = DataLoader(dataset, batch_size=64, shuffle=True,
                        num_workers=4, pin_memory=True, drop_last=True)
```

2.5 Cell 4 — model, loss, training loop

```
class PatchEncoder(nn.Module):
    def __init__(self, embed_dim=128):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(1, 32, 5, stride=2, padding=2), nn.BatchNorm2d(32), nn.ReLU(),
            nn.Conv2d(32, 64, 3, stride=2, padding=1), nn.BatchNorm2d(64), nn.ReLU(),
            nn.Conv2d(64, 128, 3, stride=2, padding=1), nn.BatchNorm2d(128), nn.ReLU(),
            nn.AdaptiveAvgPool2d(1),
        )
        self.fc = nn.Linear(128, embed_dim)

    def forward(self, x):
        z = self.fc(self.net(x).flatten(1))
        return F.normalize(z, dim=1)

def triplet_loss(a, p, n, margin=0.3):
    d_pos = 1 - (a * p).sum(dim=1)
    d_neg = 1 - (a * n).sum(dim=1)
    return F.relu(d_pos - d_neg + margin).mean()

model = PatchEncoder().to(DEVICE)
if torch.cuda.device_count() > 1:          # use both T4s
    model = nn.DataParallel(model)

opt   = torch.optim.AdamW(model.parameters(), lr=3e-4, weight_decay=1e-4)
sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=50)

EPOCHS = 50
for epoch in range(EPOCHS):
    model.train()
    running = 0.0
    for anchor, positive, negative in dataloader:
        anchor, positive, negative = (t.to(DEVICE, non_blocking=True)
                                      for t in (anchor, positive, negative))
        with torch.autocast(device_type="cuda", dtype=torch.bfloat16):
            za, zp, zn = model(anchor), model(positive), model(negative)
            loss = triplet_loss(za, zp, zn)
        opt.zero_grad(set_to_none=True)
        loss.backward()
        opt.step()
        running += loss.item()
    sched.step()
    print(f"epoch {epoch:02d}  loss {running/len(dataloader):.4f}")
    if epoch % 10 == 0:
        torch.save(
            (model.module if hasattr(model, "module") else model).state_dict(),
            f"{CKPT_DIR}/epoch{epoch}.pt"
        )
```

2.6 Cell 5 — validation (never assert on a fixed embedding, only on ranking)

```
@torch.no_grad()
```

```
def validate(model, n_trials=500):
    m = model.module if hasattr(model, "module") else model
    m.eval()
    correct = 0
    val_ds = TripletDataset(dem_files, patch=128, length=n_trials)
    for a, p, n in val_ds:
        a, p, n = a.unsqueeze(0).to(DEVICE), p.unsqueeze(0).to(DEVICE), n.unsqueeze(0).to(DEVICE)
        za, zp, zn = m(a), m(p), m(n)
        d_pos = 1 - (za*zp).sum()
        d_neg = 1 - (za*zn).sum()
        correct += (d_pos < d_neg).item()
    return correct / n_trials

acc = validate(model)
print(f"triplet ranking accuracy: {acc:.3f}  (target: comfortably > 0.5, ideally > 0.9)")
```

2.7 Cell 6 — export to ONNX (this is the only file you carry forward)

```
m = model.module if hasattr(model, "module") else model
m.eval().to(DEVICE)
dummy = torch.randn(1, 1, 128, 128, device=DEVICE)
torch.onnx.export(
    m, dummy, "/kaggle/working/descriptor.onnx",
    input_names=["patch"], output_names=["embedding"], opset_version=17,
)
print(os.path.getsize("/kaggle/working/descriptor.onnx") / 1e6, "MB")  # sanity check, expect low
single-digit MB

# Trim /kaggle/working before committing -- only the ONNX export needs to survive
import shutil
shutil.rmtree(CKPT_DIR, ignore_errors=True)
```

2.8 Turning the export into a Kaggle Dataset, then attaching it to the ARC-AGI-3 notebook

```
# In the generic notebook's own terminal, or a local machine with the Kaggle CLI + descriptor.onnx:
mkdir -p descriptor_export
cp /kaggle/working/descriptor.onnx descriptor_export/
cd descriptor_export

# metadata for the dataset (kaggle datasets init writes a template you edit):
kaggle datasets init -p .
# edit dataset-metadata.json: set "title" and "id": "<your-kaggle-username>/lunar-descriptor-v1"

kaggle datasets create -p .
# subsequent updates to the same weights:
# kaggle datasets version -p . -m "retrained, epoch 50, val acc 0.93"
```

Then, inside the ARC-AGI-3 competition notebook: **Add Input** → **Datasets** → **search your dataset id (lunar-descriptor-v1)** → **Add**. It mounts read-only at /kaggle/input/lunar-descriptor-v1/descriptor.onnx, loadable with the same onnxruntime.InferenceSession(...) call shown in Document 1 §C.4 — no retraining, no internet needed inside that session, exactly the split Document 1 already describes for data prep vs. accelerated compute, just with the training half moved to a separate notebook instead of the same one.

3. Keeping every output under the 20 GB Kaggle private-dataset cap

Kaggle enforces a 20 GB cap *per private dataset* (and a separate 20 GB cap on a notebook's own /kaggle/working output at commit time). Nothing in this project's pipeline is naturally anywhere near that limit if you follow the crop-before-upload discipline below — the actual bottleneck is almost always someone uploading a full scene instead of a patch.

Artifact	Typical size	Cap risk	Rule
Raw PRADAN OHRC/TMC-2 .img scene	1–3 GB per scene	High if you upload whole scenes	Never uploaded as a dataset — stays local/PRADAN-side; only crops leav

Cropped DEM patch (128–512 px, GeoTIFF)	~0.1–5 MB each	Negligible	This is what goes in the lunar-dem-patches-v1 dataset
Synthetic pair set (rendered from patches)	a few hundred MB for thousands of pairs	Large	Generate on the fly in Cell 3 above rather than pre-rendering to disk, unles
Checkpoint .pt files (per-epoch)	~1–3 MB each (small model)	Negligible individually, adds up over many epochs	Delete early checkpoints before committing (Cell 6); keep only the final e
Exported descriptor.onnx	1–5 MB	None	This is the only artifact that should ever leave as the training dataset
Crater catalog subset (CSV/shapefile)	a few MB regionally	Negligible	Filter to your scene's footprint before upload, per Document 3 Step 7

Practical check before any kaggle datasets create call: du -sh descriptor_export/. For this project that number should read single-digit megabytes, not gigabytes — if it doesn't, something raw got copied in by mistake.

4. Exact datasets/files to download — explicit checklist (extends Document 3)

Document 3 already names the five portals and two supporting datasets. Below is the same list collapsed into a literal checklist of what file(s) to click, so there's no ambiguity about which product on each portal you're pulling.

#	Source	Exact item to pull	Where it lands
1	PRADAN (pradan.issdc.gov.in)	PDS4-labelled product for your chosen orbit/scene ID: the paired .xml label + .img raw array (OHRC/TMC-2) or hyperspectral .c	data/raw/<instrument>/<scene_id>/
2	Chandrayaan Map Browse (chmapbrowse.issdc.gov.in)	No download here — used only to note the orbit/scene ID, lat/lon bounding box, and acquisition date that you then search for o	notes only
3	LROC NAC (lroc.im-ldi.com)	The specific NAC product ID confirmed against your footprint on QuickMap — a PDS-family ras	data/reference/lroc_nac/<product_id>/
4	LROC QuickMap (quickmap.lroc.im-ldi.com)	No download — coverage/coordinate confirmation only, before you pull anything full-resolution.	notes only
5	JAXA SELENE archive	Fallback reference product for the same footprint, only if NAC coverage is thin — Search the current data archive index (p	data/reference/selene/<product_id>/
6	LOLA gridded data (PDS Geosciences Node) or SLDEM2015	A DEM tile covering your scene's footprint — SLDEM2015 if your crater falls within ±60° latitude (higher resolution), else L	data/dem/<region_name>_sldem2015.tif
7	LPI/USGS Lunar Crater Database or Robbins (2019) catalog	Regional CSV/shapefile subset filtered to craters whose rims fall inside your scene's footprint	data/crater_catalog/<region_name>_craters.csv

Order to actually do this in (unchanged from Document 3, restated for convenience): Step 1 register on PRADAN first (longest lead time) → Step 2 browse candidates on Chandrayaan Map Browse → Step 3 confirm coverage on QuickMap → Step 4 pull the PRADAN source once approved → Step 5 pull the LROC/SELENE reference → Step 6 pull/crop the DEM → Step 7 pull/filter the crater catalog → Step 8 verify the Makharia et al. 2025 ISRO SAC citation before it goes near a slide.

For the training workflow in §2, only the DEM tile from Step 6 is actually consumed (cropped into patches, then rendered into synthetic sun-angle pairs) — the raw OHRC/TMC-2/NAC imagery from Steps 1, 4 and 5 is used later in the FastAPI /match pipeline (Document 4), not in descriptor training itself.

5. Summary of what changed vs. Documents 1 and 3

- Added: a second, equally valid training path — generic Kaggle T4x2 notebook, full internet, train + export ONNX there, then attach as a Dataset input to the ARC-AGI-3 rtx6000 notebook — alongside Document 1 Part A.3's original path of training inside the ARC-AGI-3 notebook itself.
- Added: the complete, runnable six-cell training script for that path, including DataParallel across the T4x2 pair.
- Added: an explicit per-artifact size table so every upload stays far under Kaggle's 20 GB private-dataset cap.
- Added: a literal, numbered checklist mapping each of Document 3's five portals + two supporting datasets to the exact file/product to click, and which pipeline stage actually consumes it.